1.1 What is Encapsulation ? Explain how Java achieves it using access modifiers and getter/setter methods. Give one real-life analogy.

Encapsulation - is an important concept of Object-Oriented Programming (OOP). It means wrapping data and methods together inside a single class and protecting the data from direct access from outside the class.

In Java, encapsulation is mainly achieved by -

1. Declaring data members as `private`.
2. Providing `public getter and setter methods` to access or modify the private data.

For example :

```
class Student {
    private String name;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

Here, name is private, so it cannot be directly accessed

from outside the class. The getName() method is used to read the value, and setName() is used to change the value.

Access Modifiers -

- Private → accessible only inside the same class.
- public → accessible from anywhere.
- protected → accessible within the same package and subclass.
- default → accessible within the same package.

For proper encapsulation, we usually keep data private and provide controlled access through public methods.

Real Life analogy -

A bank account is a good example of encapsulation. We cannot directly change the money stored in our bank account. Instead, we use controlled operations such as deposit and withdraw. The actual balance is protected by the bank.

In short, Encapsulation provides data hiding, security, and controlled accessed to data.

1.2. What is Polymorphism ? Distinguish compile-time (static) vs runtime (dynamic) polymorphism with one example each.

Polymorphism - is one of the main concepts of object-Oriented Programming . In Java, polymorphism means the same method name or interface can perform different behaviors depending on how it is used.

There are two main types of Polymorphism in Java :

1. Compile-time Polymorphism : Compile time polymorphism is also called static polymorphism. It is mainly achieved through method overloading.

In method overloading , multiple methods have the same name but different parameters -

Example :

```
class Calculator {
    int add (int a , int b) {
        return a+b ;
    }
    double add (Double a , double b ){
        return a+b ;
    }
}
```

2. Run-time Polymorphism - Run-time polymorphism is also called Dynamic Polymorphism. It is achieved through method overriding.

Example :

```
class Animal {
    void sound () {
        System.out.println ("Animal makes sound");
} }
class Dog extends Animal {
@Override
 void sound () {
        System.out.println ("Dog barks."); } }
```

If we write :

```
Animal a = new Dog();
a.sound();
```

The output will be: Dog barks.

Difference -

Compile-time	Run-time
Also call static Polymorphism	Also called Dynamic Polymorphism
Achieved by method overloading	Achieved by method overriding
Decision occurces at compile time	Decision occurs at Run-time
Same method name, diff. parameter.	Same method with redefined behaivior.

1.3 Write a BankAccount class demonstrating encapsulation (Private balance + getbalance() / deposit()) and Polymorphism (overloaded deposit (double) / deposite (double, String remarks)). Include a Main class calling both overloads.

```
class BankAccount {

    private double balance ;
    // Constructore
public  BankAccount (double balance) {
        this.balance = balance ;
    }
    // getter
public  double  getBalance () {
            return  balance ;
    }
    // Deposit method - 1
public void deposit (double amount) {
        balance = balance + amount ;
        System.out.println ("Deposited: " + amount);
    }
    // Deposit method - 2 (overloaded)
Public void deposit (double amount, String remarks){
        balance = balance + amount ;
        System.out.println ("Deposited: " + amount);
        System.out.println ("Remarks: " + remarks);
    }
}
```

```
public class Main {
    public static void main (String[] args) {
        BankAccount account = new BankAccount (5000);
        // 1st overloaded method call
        account.deposit(1000);
        // 2nd overloaded method call
        account.deposit(2000, "Salary");
        System.out.println("Final Balance: "+account.getBalance());
    }
}
```

This program demonstates both encapsulation and polymorphism.

Encapsulation - `private double balance;`

the balance variable is private, so it cannot be direct accessed from outside the `BankAccount` class.

We use - `public double getBalance()` - to safely read the balance

Polymorphism - There are two deposit methods -

1. deposit (double amount)
2. deposit (double amount, String remarks)

2.1. Compaire overloading vs overriding : definition, class involved , parameters , return type , binding time .

Method Overloading - means having methods with the same name but different paramet lists in the same class. It is an example of compile-time polymorphism.

Method Overriding - a child class provides its own implementation of a method that is already defined in its parent class. It is an example of ~~compile-tim~~ run-time polymorphism.

Overloading	Overriding
Same method name with diff. parameters.	Same method name and same parameters.
Occurs within same class	Requires parent and child class
Parameter must be different	Parameter must be same
Compile time binding	Runtime binding
Static Polymorphism	Dynamic Polymorphism

Example : Overloading

```
void show (int x) { }
void show (String x) { }
```

Overriding

```
class Animal {
    void sound () {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
@Override
    void sound () {
        system.out.println( "Dog barks" );
    }
}
```

2.2 Explain Early binding vs late binding. Why is overriding resolved at run time and overloading at compile time?

Binding means connecting a method call with the actual method implementation.

Early Binding - is also called static/compile-time binding when the compiler decides which method should be called before the program runs.

Method overloading uses early binding because compiler can identify the correct method from the method name and parameter list.

Example -

```
class Calculator {
    void add (int a, int b) {
        System.out.println (a+b);  }
    void add (double a, double b) {
        System.out.println (a+b);  }}
```

Late Binding - also called dynamic/run-time binding. It is the actual method to execute is decided during program execution.

Method Overriding uses late binding because the actual object may be a child-class object even

when the reference is of the parent type.

Example -

```
Animal a = new Dog();
a.sound();
```

Here, `a` is an `Animal` reference, but the actual object is `Dog`. Therefore, Java calls `Dog`'s `sound()` method at run-time.

2.3 Shape with area(), overriden by Circle and Rectangle (late binding via shape ref.). Add Overloaded describe(str / describe(string, int) in shape (early binding). Show example output.

```
class Shape {
  public void area() {
    System.out.println("Area of Shape");
  }
  public void describe(String name) {
    System.out.println("Shape : " + name);
  }
  public void describe(String name, int sides) {
    System.out.println("Shape :" + name);
    System.out.println("Number of sides: " + sides);
  }
}
class Circle extends ~~circle~~ Shape {
  @Override
  public void area() {
    double r = 5;
    System.out.println("Circle Area : " + Math.PI
                       * r * r);
  }
}
class Rectangle extends Shape {
  @Override
  public void area() {
    double length = 5;
```

```
    double width = 4;
    System.out.println("Rectangle Area: " + length
                                    * width);  }  }
public class main {
  public static void main (String[] ads){
     Shape s1 = new Circle();
     Shape s2 = new Rectangle();
  // late binding
     Shape s1.area();
          s2.area();

  // Early binding
     s1.describe("Circle: ");
     s2.describe("Rectangle", 4);   }  }
```

Sample Output -

```
Circle Area: 78.53981633974483
Rectangle Area: 20.0
Shape : Circle
Shape : Rectangle
Number of Sides : 4
```

3.1 Define abstract class and interface. List ≥4 structural differences fields, constructors, method bodies, multiple inheritance)

Abstract Class- is a class declared using `abstract` keyword. It can contain both abstract method and concrete method.

Example:

```
abstract class Vehicle {
    String brand ;
  Vehicle (String brand){
        this.brand = brand; }
 void showBrand (){
    System.out.println ("Brand" + brand);}
  abstract void start ();
}
class Car extends Vehicle {
    Car (String brand) {
      super (brand);
                }
void start (){
    System.out.println ("Car starts with a key."); }
                                                    }
public class Main {
   public static void main (String[] ads){
   Car c = new Car ("Toyota");
     c.showBrand();
     c.start() ;
                    } }
```

Interface - is a contract that specifies what a class should do. A class uses the **implements** keyword to implement interface.

Example -

```
interface Vehicle {
    void start ();  }
class Car implements Vehicle {
    public void start () {
        system.out.println ("Car starts with a key");
}  }
Public class Main {
    public static void main (String[] ads) {
        Car C = new Car ();
        c.start ();
                    } }
```

Difference -

	Abstract Class	Interface
Fields	Can have instance, static mutable or final fields	Fields are implicity public static final (const. only)
Constructor	Can have constructor to initialize objec state.	Can't have constructor
Method bodies	both abstract (without body) or concrete (with body) methods.	Traditionally only abstract method. Though modern version allow default and static method.
Multiple inheritance	A class can extend only one abstract class (single inheritance)	Can implement multiple inheritance.

3.2 When should you choose an abstract class vs interface? Give two real world scenarios ("is-a" with shared code vs "can-do" capability)

Abstract class - we choose when related classes have:

- A common parent
- Common fields
- Common methods / code.
- An "is-a" relationship.

Real-world example:

Cars, Bus and Truck are all Vehicles

So, the Vehicle class can provide common code such as startEngine().

Interface - we choose when different classes need to have the same capability, even if they are not closely related.

Real-world example-

Car → Insurable
House → Insurable
Phone → Insurable ; they are not same type of object, but they can have an Insurable capability

In short:

- Abstract → "What you are" / is-a
- Insurance → "What you can do" / can-do / has-a

3.3 Vehicle (abstract, startEngine() implemented + abstract fuelType()) and Insurable (interface, calculatePremium()). Car extends Vehicle and implements Insurable. Comment on why each construct was choosen.

```
abstract class Vehicle {
  public void startEngine() { // concrete method
    system.out.println("Engine started.");
  }
  public abstract void fuelType(); // Abstract method
                                   (no body)
}

interface Insurable {
    void calculatePremium();
  }
 class Car extends Vehicle implements Insurable{
@Override
   public void fuelType() {
     System.out.println("Fuel Type : Petrol");
  }
  @Override
     public void calculatePremium() {
       System.out.println("Insurance premium: 5000");
}}
```

```
public class Main {
    public static void main (String[] args) {
        Car car = new Car ();
        car.startEngine ();
        car.fuelType ();
        car.calculatePremium ();
    }
}
```

Output-

Engine started.
Fuel Type : Petrol
Insurance Premium : 5000

Why abstract class -

Car is a type of Vehicle, so there is an is-a relationship.

Vehicle also provides common behavior:

`startEngine()`

while forcing subclasses to define:

`fuelType().`

Why interface - Insurable represents capability.

A car can be insured, so Car implements Insurable

So

Car `is a` Vehicle + Car `has` Insurable capability